

[Home](#) > [API reference](#) > [DataFrame](#) > [pandas.DataFrame](#)...

pandas.DataFrame.rename

`copy=None, inplace=False, level=None, errors='ignore')`[\[source\]](#)

Rename columns or index labels.

Function / dict values must be unique (1-to-1). Labels not contained in a dict / Series will be left as-is. Extra labels listed don't throw an error.

See the [user guide](#) for more.

Parameters:

mapper : *dict-like or function*

Dict-like or function transformations to apply to that axis' values. Use either `mapper` and `axis` to specify the axis to target with `mapper`, or `index` and `columns`.

index : *dict-like or function*

Alternative to specifying axis (`mapper, axis=0` is equivalent to `index=mapper`).

columns : *dict-like or function*

Alternative to specifying axis (`mapper, axis=1` is equivalent to `columns=mapper`).

axis : {0 or 'index', 1 or 'columns'}, default 0

Axis to target with `mapper`. Can be either the axis name ('index', 'columns') or number (0, 1). The default is 'index'.

copy : *bool, default True*

Also copy underlying data.

[Skip to main content](#)

 Note

The `copy` keyword will change behavior in pandas 3.0. [Copy-on-Write](#) will be enabled by default, which means that all methods with a `copy` keyword will use a lazy copy mechanism to defer the copy and ignore the `copy` keyword. The `copy` keyword will be removed in a future version of pandas.

You can already get the future behavior and improvements through enabling copy on write `pd.options.mode.copy_on_write = True`

inplace : *bool, default False*

Whether to modify the DataFrame rather than creating a new one. If True then value of copy is ignored.

level : *int or level name, default None*

In case of a MultiIndex, only rename labels in the specified level.

errors : *{'ignore', 'raise'}, default 'ignore'*

If 'raise', raise a `KeyError` when a dict-like `mapper`, `index`, or `columns` contains labels that are not present in the Index being transformed. If 'ignore', existing keys will be renamed and extra keys will be ignored.

Returns:**DataFrame or None**

DataFrame with the renamed axis labels or None if `inplace=True`.

Raises:**KeyError**

If any of the labels is not found in the selected axis and "errors='raise'".

 See also[DataFrame.rename_axis](#)

Set the name of the axis.

Examples

`DataFrame.rename` supports two calling conventions

- `(index=index_mapper, columns=columns_mapper, ...)`

[Skip to main content](#)

We *highly* recommend using keyword arguments to clarify your intent.

Rename columns using a mapping:

```
>>> df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
>>> df.rename(columns={"A": "a", "B": "c"})
```

	a	c
0	1	4
1	2	5
2	3	6

Rename index using a mapping:

```
>>> df.rename(index={0: "x", 1: "y", 2: "z"})
```

	A	B
x	1	4
y	2	5
z	3	6

Cast index labels to a different type:

```
>>> df.index
RangeIndex(start=0, stop=3, step=1)
>>> df.rename(index=str).index
Index(['0', '1', '2'], dtype='object')
```

```
>>> df.rename(columns={"A": "a", "B": "b", "C": "c"}, errors="raise")
Traceback (most recent call last):
KeyError: ['C'] not found in axis
```

Using axis-style parameters:

```
>>> df.rename(str.lower, axis='columns')
```

	a	b
0	1	4
1	2	5
2	3	6

```
>>> df.rename({1: 2, 2: 4}, axis='index')
```

	A	B
0	1	4
2	2	5

Created using [Sphinx](#) 7.2.6.